Introduction to Python as a Calculator

- **Overview:**
  - Python can be used as a powerful calculator to perform basic arithmetic operations similar to a standard graphing calculator.
  - Operations include addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (**).
- **Order of Operations:**
  - Python follows the standard mathematical order of operations: Parentheses, Exponents, Multiplication and Division (from left to right), Addition and Subtraction (from left to right).
  - Parentheses can be used to change the order of execution.
- **Example Calculation:**

```python
(3*4)/(2**2 + 4/2)  # Output: 2.0
```

Using Python's Math Module

- **Accessing Mathematical Functions:**
  - Python provides various arithmetic functions through the math module, which needs to be imported using import math.
  - Common functions include:
    - math.sqrt(x) - Square root
    - math.sin(x) - Sine function
    - math.exp(x) - Exponential function
    - math.log(x) - Natural logarithm
    - math.pi - Value of π
- **Example Calculations:**

```python
import math
math.sin(math.pi/2)  # Output: 1.0
math.exp(math.log(10))  # Output: 10.000000000000002
```

- **Tips:**
  - Use TAB completion in Jupyter notebooks or Ipython to auto-complete function names.
  - View function documentation by appending a question mark ? after the function name, e.g., math.factorial?.

Handling Different Data Types and Complex Numbers

- **Data Types in Python:**
  - **int**: Integer values (e.g., 1, 42)
  - **float**: Floating-point numbers (e.g., 3.14, 2.718)
  - **complex**: Complex numbers (e.g., 2 + 5j)
- **Working with Complex Numbers:**
  - Python uses j to denote the imaginary unit.
  - You can create complex numbers directly or using the complex() function.

```python
2 + 5j  # Output: (2+5j)
complex(2, 5)  # Output: (2+5j)
```

- **Scientific Notation:**
  - Python supports scientific notation for large numbers, using e to denote powers of 10.

```python
3e0*3.65e2*2.4e1*3.6e3  # Output: 94608000.0
```

Introduction to Managing Python Packages

Key Points:

- **Importance of Packages in Python:**

Python's strength lies in the vast number of community-developed packages/modules that provide pre-coded functions and algorithms.

- **Role of Package Management:**

Managing packages efficiently is crucial to fully leverage Python's capabilities, as it allows for easy installation, updating, and removal of packages.

- **Primary Tool - pip:**

pip is the most common package manager used in Python, automating the process of package management.

Common pip Commands

Key Commands:

- **Installing a Package:**
  - Command: pip install package_name
  - To install a specific version: pip install package_name==1.5
- **Upgrading a Package:**
  - Command: pip install --upgrade package_name
  - Shortcut: pip install -U package_name
- **Uninstalling a Package:**
  - Command: pip uninstall package_name
- **Listing Installed Packages:**
  - Command: pip list
- **Displaying Package Information:**
  - Command: pip show package_name

Advanced Package Management

Installing Packages from Source:

- **When to Use:**

Occasionally, a package may not be available on PyPI, requiring manual installation fr
source files.

- **Process:**
  - Download and uncompress the source file.
  - Navigate to the directory containing the setup.py script.
  - Run the command: python setup.py install
- **Considerations for Windows Users:**

Use a command prompt window to execute the installation command.

Introduction to Jupyter Notebook

Overview:

- **Jupyter Notebook** is an open-source web application that allows for the creation and sharing of documents containing live code, equations, visualizations, and narrative text.
- Commonly used for:
  - Data cleaning and transformation
  - Numerical simulation
  - Statistical modeling
  - Data visualization
  - Machine learning

Advantages:

- Combines code and rich text elements such as equations, links, images, and tables in one document.
- Facilitates the creation of interactive content like blogs, papers, and books.
- Ideal for sharing ideas and results with others.

Working with Jupyter Notebooks

Starting a Jupyter Notebook:

- Run the command jupyter notebook in your terminal to launch the notebook interface in your default web browser.
- The interface displays a dashboard showing files, running notebooks, and options to create new notebooks.

Notebook Interface:

- The interface includes a toolbar and a menu that provide access to various tools and functions.
- **Key Features:**
  - **Cells:** The basic unit where you can write and execute code or add text.
  - **Cell Types:**
    - **Code cells** for writing and running Python code.
    - **Markdown cells** for adding descriptive text, using rich text formatting.

Basic Operations:

- Execute a cell by pressing Shift + Enter.
- Cells can be rearranged, inserted, or deleted as needed.

Managing and Closing Jupyter Notebooks

Shutting Down a Notebook:

- Simply closing the browser tab does not stop the notebook; the server continues running in the background.
- To fully shut down a notebook, close the associated terminal where it was launched.

Proper Closure:

- To completely close a notebook:
  - Go to the dashboard.
  - Check the box next to the running notebook.
  - Select the shutdown option from the toolbar.

Conclusion:

- With the basics covered, you're ready to start using Jupyter Notebook to develop and share Python projects.

Introduction to Logical Expressions and Operators in Python

Logical Expressions:

- A logical expression evaluates to either True or False.
- Python uses the boolean data type for logical expressions.
  - True is equivalent to 1
  - False is equivalent to 0
- **Example:**
  - Expression: 3 < 4
  - Python evaluates it as True.

Comparison Operators:

- Used to compare values in logical expressions.
- Key Operators:
  - > : Greater than
  - >= : Greater than or equal to
  - < : Less than
  - <= : Less than or equal to
  - != : Not equal
  - == : Equal

Example of Comparison:

- Given a = 4 and b = 2:
  - a > b evaluates to True
  - a == b evaluates to False

Logical Operators and Order of Operations

Logical Operators:

- Operate on two or more logical expressions.
- **Key Operators:**
  - and: True if both expressions are true.
  - or: True if at least one expression is true.
  - not: Inverts the truth value of an expression.

Truth Table Examples:

- **AND Operator:**
  - P and Q is True if both P and Q are True.
- **OR Operator:**
  - P or Q is True if at least one of P or Q is True.

Order of Operations:

- Arithmetic operations are executed first, followed by comparison operations, and finally logical operations.
- **Tip:**
  - Use parentheses to clarify order, e.g., (1 + 3) > (2 + 5) is more readable.

Python Quirks:

- Python allows expressions like (3 > 2) + (5 > 4), which resolves to 2.
- Any non-zero number is considered True in logical operations.